

Using R with LLMs

Tuesday, June 23, 2026

i Today: Tuesday June 23

Before class

- **Perusall Reading** (Canvas): *Data Computing* §18.3 (trees)

Plan for today

- Three ways to use a **large language model (LLM)** with R: **chat**, **in the IDE**, and **from code**.
- How to **prompt well** for R help.
- Calling an LLM **from R** with the **ellmer** package.

Helpful links

- [AI policy](#) · [ellmer](#) · [reprex](#) · [Syllabus](#) · [Schedule](#)

Overview

LLMs are part of the R workflow now

An **LLM** (ChatGPT, Claude, Gemini, etc.) predicts likely text from a prompt. For an R user that's useful for:

- **Explaining errors**: paste a cryptic message, get a plain-English cause.
- **Drafting code**: “write a `ggplot` of X by Y, faceted by Z.”
- **Refactoring**: “make this tidyverse instead of base R.”
- **Learning**: “what does `across()` do, with a small example?”
- **As a data tool itself**: classify, extract, or summarize text *inside* a pipeline.



Tip

The skill that matters most is **not** “getting an answer.” It’s judging whether the answer is **right**.

Three places you’ll use one

Where	Examples	Best for
Chat window	Claude, ChatGPT, Gemini	one-off questions, explaining errors, learning
In the IDE	Positron / RStudio assistants, GitHub Copilot	inline autocomplete, “fix this” on selected code
From R code	the ellmer package	doing the <i>same LLM call on every row</i>

Prompting well for R

Be specific

A vague prompt gets a vague (often wrong) answer. A good R prompt includes:

1. **The goal:** what the output should be.
2. **The data shape:** paste `str()`, `glimpse()`, or `dput(head(x))`.
3. **Anything you tried** and (if applicable) the **exact error**.
4. **Constraints:** “use the tidyverse,” “base R only,” “no new packages.”



Warning

Don’t paste private or sensitive data. Anything you send leaves your machine and may be stored. Share a tiny **fake** example with the same structure, not the real dataset.

A reproducible example beats a paragraph

The single most useful thing you can hand an LLM (or a human, or Stack Overflow) is a **reprex**, which is a tiny, self-contained, runnable snippet that reproduces the problem.

```
# install.packages("reprex")
library(reprex)

# copy this to your clipboard, then run reprex()
# it runs the code and formats code + output together that's ready to paste:
reprex({
  library(dplyr)
  tibble(x = c(1, 2, NA)) |> summarise(m = mean(x)) # why is m NA?
})
```

Built-in datasets (`mtcars`, `iris`, `penguins`) make great reprexes. This helper can run them too.

Weak prompt vs. strong prompt

Weak

“my ggplot doesn’t work, fix it”

You’ll get a generic guess that may not match your situation.

Stronger

“Using `penguins`, I want a scatter of `bill_length_mm` vs `flipper_length_mm` colored by `species`. My code `ggplot(penguins) + geom_point(bill_length_mm, flipper_length_mm)` errors with ‘*object not found*’. Use tidyverse to tackle this.”

Goal + data + code + error + constraint.

Calling an LLM from R: `ellmer`

What `ellmer` is

`ellmer` is the **tidyverse package** that provides a unified interface for accessing various LLMs from R:

- `chat_anthropic()`: Claude
- `chat_openai()`: GPT
- `chat_google_gemini()`: Gemini

```
# install.packages("ellmer")
library(ellmer)

chat <- chat_anthropic(
  model = "claude-opus-4-8", # current default Claude model (RIP Fable 5)
  system_prompt = "You are a concise R tutor. Prefer the tidyverse."
)

chat$chat("In one sentence, what does dplyr::across() do?")
```

Note

The newest, most capable Claude models are **Fable 5** (claude-fable-5) and the **Claude 4.x** family: **Opus 4.8** (claude-opus-4-8), **Sonnet 4.6** (claude-sonnet-4-6), **Haiku 4.5** (claude-haiku-4-5). Bigger models are smarter but cost more per call.

API keys: never in your code

Calling a provider needs an **API key** (a secret password tied to a paid account).

```
# DON'T: a key pasted in a script gets committed to GitHub forever.
chat_anthropic(api_key = "sk-ant-REAL-KEY")

# INSTEAD: store it once in an environment variable ellmer can read automatically.
usethis::edit_r_environ() # opens ~/.Renviron; add: ANTHROPIC_API_KEY=sk-ant-...
# restart R, then:
chat <- chat_anthropic(model = "claude-opus-4-8") # key picked up from the env
```

Warning

A leaked key can run up real charges. Keep it out of `.R`, `.qmd`, and anything you push to git (`.Renviron` is git-ignored by default).

LLM as a *data step*

The real power for us isn't chatting, but rather running the **same prompt over every row** of a data frame. This approach can help turn messy text into structured columns since `ellmer` can return **typed, structured output**:

```
library(ellmer)
chat <- chat_anthropic(model = "claude-opus-4-8")

reviews <- tibble(text = c(
  "Loved it, arrived early and works great!",
  "Broke after two days. Very disappointed.",
  "It's fine. Does the job, nothing special."
))

# ask for a schema; ellmer enforces the shape
sentiment <- type_object(
  mood = type_string("one of: positive, negative, neutral"),
  reason = type_string("a 5-word justification")
)

reviews |>
  mutate(parsed = map(text, ~ chat$chat_structured(.x, type = sentiment))) |>
  unnest_wider(parsed)
```

This is `purrr` + a prompt: text in, tidy columns out. *That's* the data-science use.

The catch

LLMs are confident, not always correct

An LLM generates **plausible**, not 100% true, text. You should expect:

- **Hallucinated functions / packages** like `dplyr::summarise_with()` that don't exist.
- **Subtly wrong code** that runs without error, gives the wrong number.
- **Outdated idioms**, i.e., deprecated `fun()`, old `gather()/spread()`.
- **Made-up facts and citations** stated with total confidence.

Verification

For any code an LLM gives you:

Tip

1. **Read it.** Can you explain every line?
2. **Run it.** Does it error on real input data? Does it even do what you asked?

3. **Check the result.** Spot-check against a value you can compute by hand or already know.
4. **Test the edge cases.** Things to try include empty input, `NA`s, one row, or a weird category.

Use it wisely

- **Learn from it, don't just paste.** Ask *why*, ask for a small example, then reproduce it yourself.
- **Keep prompts small and specific.**
- **Prefer reprexes and built-in data.**
- **Track what you used.**

Note

Treat LLMs like very fast, occasionally-wrong collaborators. They're not to be blindly trusted.

Activity

In-class activity (~20 min)

Use whatever LLM you have access to (chat window is fine, no API key needed).

1. **Explain an error.** Run this, then ask the LLM to explain *and* fix it. Verify the fix runs:

```
library(dplyr)
penguins |> group_by(species) |> summarize(avg = mean(bill_length_mm))
```

(Why `NaN/NA`? What's the one-argument fix?)

2. **Generate a function.** Ask for an R function that takes a numeric vector and returns its mean, median, and SD as a named tibble. Then **break it**: feed it `c(1, 2, NA)` and a length-1 vector. Did the LLM handle those?
3. **Catch a mistake.** Find **at least one** thing the LLM got wrong, outdated, or could improve.

Wrap-up & looking ahead

What we covered

- Three surfaces: **chat**, **IDE assistant**, **ellmer from code**.
- **Prompt well**: goal + data shape + error + constraints; hand over a **reprex**, never private data.
- **ellmer** calls Claude/GPT/Gemini from R; keep API keys in `.Renvi`; structured output = LLM as a tidy **data step**.

Upcoming

- **Wed Jun 24**: benzoyl peroxide case study: data-driven investigation uncovering carcinogens in everyday skincare products · Checkpoint 2 due

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

- [ellmer docs](#) · [reprex](#)
- [AI policy](#) · [Syllabus](#) · [Schedule](#)